

la ia como herramienta seria para un des

Investigación hecha con IA (Claude). Documento sobre usar modelos de IA como herramienta seria en un flujo de desarrollo web freelance: modelos grandes vs locales, y el experimento de fine-tuning "Frankenstein".

TL;DR

- Un modelo "mejor" suele ser más grande porque las **leyes de escalado** dicen que más parámetros + más datos + más cómputo = menos errores; pero los modelos frontera (cientos de miles de millones de parámetros) necesitan cientos de GB o TB de memoria y muchas GPUs en un centro de datos. Un MacBook Air M5 con 16-32 GB solo corre modelos pequeños (hasta ~30B con cuantización), no los grandes.
- Para trabajo web real (HTML/CSS/JS a mano, sitios estáticos), los modelos locales gratis en Ollama (Qwen3-Coder 30B, Qwen2.5-Coder, Llama 3.x) bastan para bocetos, snippets, autocompletado y tareas repetitivas. Para razonamiento largo, proyectos multi-archivo y gusto estético, Claude Sonnet/Opus siguen siendo claramente superiores.
- División práctica: **Haiku** para lo mecánico, **Sonnet** como caballo de batalla, **Opus/Fable** para arquitectura y depuración difícil.
- Para el experimento "Frankenstein" (100 webs -> 5 propuestas nuevas), lo más efectivo NO es entrenar nada: usa contexto largo / Projects / few-shot con un modelo grande. El fine-tuning LoRA con MLX en tu Mac es un experimento divertido que aprenderá tu estilo pero no te dará inteligencia, y con 100 ejemplos sobreajusta rápido. Entrenar desde cero es inviable (millones de dólares).

Key findings

1. **Los parámetros son los "mandos" ajustables del modelo.** Un modelo de 70.000 millones de parámetros tiene 70.000 millones de números afinados en el entrenamiento. La regla de Chinchilla (Hoffmann et al., DeepMind, 2022) fija ~20 tokens de entrenamiento por parámetro para el óptimo de cómputo.
2. **Las leyes de escalado explican por qué "más grande" = "mejor".** El error baja de forma predecible al subir parámetros y datos, pero con rendimientos decrecientes (ley de potencia): cada mejora cuesta cada vez más cómputo.
3. **La familia Claude (Haiku -> Sonnet -> Opus -> Fable) refleja esto.** Más grande = más capaz, más caro y más lento. Precios: Haiku 4.5 ~\$1/\$5 por millón de tokens, Sonnet ~\$3/\$15, Opus 4.8 \$5/\$25, Fable 5 \$10/\$50.
4. **Los modelos grandes NECESITAN centros de datos.** Un 70B necesita ~140 GB solo

para los pesos en FP16; los frontera (a GPT-4 se le atribuye ~1 billón de parámetros, sin confirmar) necesitan cientos de GB o TB repartidos entre muchas GPUs. Un MacBook Air M5 tiene 16-32 GB.

5. **Entrenar es mucho más caro que usar (inferencia).** Sam Altman confirmó que entrenar GPT-4 costó ">\$100 millones"; se estima ~25.000 GPUs A100 durante ~100 días. Usar el modelo (inferencia) es lo que haces con Claude Code o con modelos locales pequeños en tu Mac.
6. **Los modelos locales han cruzado un umbral útil.** Qwen3-Coder 30B (arquitectura MoE, 30,5B totales pero solo ~3,3B activos por token) corre bien en un Mac de 32 GB a decenas de tokens/segundo y da buena calidad para código. Pero sigue por detrás de Sonnet/Opus en las tareas difíciles.
7. **Para tu experimento, RAG/contexto largo > fine-tuning para 100 ejemplos.** El fine-tuning ajusta estilo, no conocimiento ni razonamiento; con 100 ejemplos sobreajusta rápido. LoRA con MLX es viable (un 8B se afina en ~1 hora en 16 GB) pero para inspiración creativa el contexto largo de un modelo grande rinde más.

Por qué un mejor modelo suele ser uno más grande

Cada parámetro es como una perilla de una gigantesca mesa de mezclas: un número que el modelo ajusta en el entrenamiento. Un token es el trozo de texto que lee y escribe. Durante el entrenamiento el modelo lee tokens y gira las perillas para reducir errores; en inferencia las perillas están fijas.

La regla de Chinchilla (Hoffmann et al., 2022) dice ~20 tokens por parámetro para el óptimo de cómputo. En la práctica se entrena con muchos más: Llama 3 se preentrenó con "más de 15 billones de tokens", porque interesa un modelo más pequeño (barato de usar) aunque cueste más entrenarlo.

Analogía de las leyes de escalado: estudiar para un examen tienes tres recursos -- tamaño de cerebro (parámetros), cuántos libros lees (datos) y cuántas horas estudias (cómputo). Si los subes de forma equilibrada, la nota sube de forma predecible. Pero son rendimientos decrecientes: para bajar el error a la mitad puede que necesites 10x más de todo.

Memoria: para funcionar, todos los pesos deben estar cargados en memoria a la vez. En FP16 (2 bytes/parámetro) un 70B pide ~140 GB; con cuantización a 4 bits baja a ~40 GB. Un frontera de cientos de miles de millones es imposible en una máquina de consumo. Además importa el ancho de banda de memoria (el MacBook Air M5 da 153 GB/s de memoria unificada). El Neural Engine acelera modelos pequeños que quepan en RAM, pero no hace que un 400B quepa en 32 GB.

Técnicas que cambian el cálculo: cuantización (menos bits por peso, ~3,5x menos memoria a 4 bits), destilación (un modelo grande "profesor" enseña a uno pequeño),

Mixture-of-Experts (MoE: muchos expertos y un enrutador que activa solo unos pocos por token), y calidad de datos + RLHF (un modelo bien entrenado y alineado puede superar a uno más grande peor entrenado).

Modelos locales gratuitos para tu trabajo web

Regla a 4 bits: ~0,5-0,7 GB de RAM por cada mil millones de parámetros, más margen para sistema y contexto. Modelos concretos vía Ollama:

- **Qwen3-Coder 30B (A3B / "Flash")**: probablemente tu mejor opción local. MoE, ~3,3B activos, muy rápido; la build Q4_K_M pesa ~19 GB (mejor en 32 GB).
- **Qwen2.5-Coder (7B, 14B, 32B)**: el 7B ideal para autocompletado; el 32B rinde comparable a GPT-4o en benchmarks de código pero es lento.
- **Llama 3.x** (8B general; 70B solo con 64 GB), **DeepSeek-R1 destilados** (7B/14B, buenos para razonar), y Gemma/Mistral/Codegemma/Starcoder2 (ligeros, para autocompletado).

Los locales manejan bien el 70-80% de las tareas cotidianas de código: bocetos, snippets vanilla, autocompletado, tareas mecánicas, documentación. Se quedan cortos frente a Sonnet/Opus en razonamiento largo, coherencia multi-archivo, instrucciones matizadas, gusto estético y frameworks poco comunes.

Herramientas: Ollama (API local en localhost:11434), LM Studio (GUI), Continue.dev (extensión para VS Code/JetBrains), Zed (editor en Rust con Ollama nativo), Aider (CLI para repos). Consejo: usa modelos "FIM" (fill-in-the-middle) como qwen2.5-coder o starcoder2 para autocompletado; los de chat genéricos dan mal resultado ahí.

Qué tarea darle a cada Claude

- **Haiku 4.5** (\$1/\$5): tareas mecánicas -- renombrar, convertir formatos, extraer datos, clasificar, primeros borradores, lectura de archivos como sub-agente.
- **Sonnet** (\$3/\$15): tu caballo de batalla -- la mayoría del código diario, componentes, refactors, revisión, uso de herramientas.
- **Opus 4.8** (\$5/\$25): arquitectura, decisiones de diseño, proyectos multi-archivo, depuración difícil, planificación de sesiones largas.
- **Fable 5** (\$10/\$50): solo cuando la capacidad extra cambia el resultado; ten Opus/Sonnet como fallback más barato.

El output domina la factura (Fable cuesta 10x el output de Haiku). Todos menos Haiku tienen ventana de 1M de tokens (Haiku, 200K). El prompt caching ahorra hasta el 90% en tokens de entrada repetidos (system prompt, ejemplos, referencias estables) -- clave si reutilizas tu librería de componentes. El Batch API da 50% de

descuento para trabajo no urgente. Truco de estilo del Claude Cookbook: Claude tiende a converger en diseños genéricos ("AI slop") salvo que le pidas explícitamente tipografía, color y movimiento con criterio.

El experimento "Frankenstein": 100 webs -> 5 propuestas nuevas

Opción A (recomendada) -- no entrenar nada: RAG / contexto largo / few-shot. Casi con seguridad la más efectiva para 100 ejemplos. El fine-tuning enseña estilo, no conocimiento ni razonamiento; RAG y el contexto largo "inyectan" ejemplos que el modelo combina con su inteligencia intacta. Cómo: usa un modelo grande con contexto largo (o Projects), dale 5-15 de tus mejores webs como few-shot, y pídele que combine patrones. Ventajas: cero coste, cero sobreajuste, cambias los ejemplos al instante. Escribe un DESIGN.md con tu lenguaje visual y pásalo como contexto.

Opción B -- fine-tuning de un open-weight pequeño (LoRA/QLoRA con MLX). Buen proyecto de aprendizaje. MLX (framework de Apple para Apple Silicon) da LoRA con un solo comando: un 8B se afina en ~1 hora en 16 GB; un 3B a 4 bits con 90 ejemplos tardó 3-4 minutos con ~5 GB de pico. LoRA congela el modelo base y entrena matrices pequeñas (<1% de los parámetros); QLoRA añade cuantización a 4 bits. mlx-lm espera JSONL (formatos chat/tools/completions/text); divide cada web en secciones (nav, hero, tarjetas, footer) para multiplicar ejemplos, usa --mask-prompt para calcular la pérdida solo sobre el código. El paper WebSight (arXiv 2403.09029) halló que ejemplos sintéticos y limpios rinden mejor que HTML real desordenado. ¿100 webs bastan? Es poco: el mínimo práctico de estilo suele citarse en 100-500; por debajo el modelo memoriza en vez de generalizar (el survey PEFT, arXiv 2504.14117, lo confirma). Usa early stopping. Qué esperar: aprende tu estilo y formato, NO gana inteligencia nueva.

Opción C -- entrenar desde cero: inviable. Cuesta millones de dólares, billones de tokens y miles de GPUs. No es opción para un individuo.

Licencias (importante con webs de clientes): en la UE/España el autor es el primer titular del copyright; el pago del cliente no transfiere derechos salvo cesión por escrito. Muchos contratos freelance incluyen "work for hire" o cesión total. Lo más seguro: entrena/inspira solo con webs tuyas o con permiso explícito.

Recomendaciones

Fase 0 (ahora, sin coste): sigue con Claude Code + Sonnet como caballo de batalla; Haiku para volumen mecánico, Opus para arquitectura y depuración, Fable solo si justifica el precio. Activa prompt caching. Añade a tu CLAUDE.md instrucciones anti-"AI slop".

Fase 1 (Frankenstein sensato): empieza por la Opción A -- crea un Project con 10-15 de tus mejores webs propias y pide variaciones combinatorias. Estructura tu librería de bloques en un DESIGN.md como ejemplos few-shot.

Fase 2 (local para lo cotidiano): en Ollama, Qwen3-Coder 30B (con 32 GB) o Qwen2.5-Coder 7B/14B para el Air. Conecta un modelo local al editor con Continue.dev o Zed.

Fase 3 (aprender fine-tuning, opcional): prueba LoRA con mlx-lm sobre Qwen2.5-Coder 3B/7B usando SOLO tus webs propias, con early stopping. Evalúa honestamente: si solo imita tu estilo pero no propone nada creativo, confirmarás que la Opción A era el camino.

Umbrales que cambian la recomendación: con un Mac de 64 GB+ correr un 70B local o afinar un 32B se vuelve práctico; si tu factura de Claude supera el coste de configurar local, mueve tareas repetitivas a Ollama; si necesitas privacidad (código de cliente), usa local por defecto.

Cautelas

Precios y modelos cambian rápido (los de Claude son de mediados de 2026; Sonnet 5 salió el 30 de junio de 2026 con precio introductorio -- verifica en la web oficial). Anthropic no publica el número de parámetros de sus modelos: "nombre más grande = modelo más grande" es una inferencia razonable, no un dato confirmado. Las cifras de RAM y tokens/segundo son órdenes de magnitud, no garantías. Los datos de entrenamiento de GPT-4 provienen de declaraciones y estimaciones (Semafor, Stanford AI Index), no de cifras oficiales de OpenAI. El "mínimo de 1.000 ejemplos" para fine-tuning es una regla general, no ciencia establecida. El aspecto legal es una guía general, no asesoramiento jurídico.